

NID Barcode Studio

A local pipeline for image preparation, barcode decoding and lossless export

Recommendation

Use **OpenCV + ZXing-C++ Python bindings + Tkinter**. OpenCV prepares the image; ZXing reads the barcode; Tkinter shows live input, processing candidates and results. This matches the requested ZXing workflow without introducing a Java runtime.

ZXing-C++ explicitly supports PDF417 and several other barcode families [1, 2]. OpenCV's dedicated barcode decoder documents EAN/UPC retail support, so it is not a PDF417 replacement [3]. This establishes format suitability, not a measured accuracy advantage on Bangladeshi NIDs.

What the project implements

Image files and camera input; EXIF orientation handling; grayscale and size normalization; heuristic card cropping and perspective correction; barcode-region proposals; denoising, contrast enhancement, thresholding and rotation candidates; structured result display; exact-byte preservation in CSV and JSON.

Evidence boundary

No real NID sample images or authoritative NID payload schema were supplied. The software is designed to attempt PDF417, QR Code, Data Matrix and Code 128. It does not assert that every Bangladesh card generation uses the same format or field layout. Synthetic tests establish software behavior, not production read rates.

A successful barcode read is not identity or document authentication. Explicit payload labels are displayed as unverified fields. Opaque or encrypted content remains raw; decoding does not decrypt it.

Local by design

No runtime network service or cloud upload is used. Images remain in memory unless already present as input files. Exported records contain plaintext payloads and source paths, so the output directory should be protected according to the operator's data-handling requirements.

How the pipeline works

Stage	Purpose and limitation
1. Acquire & normalize	Apply EXIF orientation to files, cap the processing dimension and convert to grayscale. Very large files are rejected before conversion.
2. Early decode	Try the full frame first. Native rotation, downscale and inversion searches handle common variations without unnecessary filtering.
3. Geometry	Try a large convex card boundary and perspective warp, then gradient-based barcode crops with margins. Keep the full-frame fallback because proposals are heuristic.
4. Recovery candidates	Try denoising, CLAHE, Otsu/adaptive thresholds, enlargement and +/-12 degree rotations. Native right-angle rotation search remains enabled.
5. Decode & inspect	Stop at the first successful candidate. Return its detected symbols and draw their bounds in that candidate's coordinate system.
6. Parse & export	Parse explicit labels only. Preserve raw bytes as Base64 and save matching UUID-named CSV and JSON files, including scan metadata.

Realtime behavior

Camera acquisition and barcode processing use background threads; Tk draws the latest captured frame and candidate previews. Capture can continue while decoding is busy. Processing cadence depends on resolution, CPU and image quality. Stop takes effect between native decode operations; it is not a hard realtime interrupt.

Lossless does not mean human-readable

Base64 preserves the exact bytes ZXing returns, including binary content. Display text can reflect ZXing's character-set handling. JSON is UTF-8; CSV uses a BOM and spreadsheet-safe text escaping. CSV nested values are JSON strings. Each file is atomic; a failure between formats may leave one member of a pair until retry.

Alternatives & tradeoffs

Java ZXing

The original ZXing project is a viable alternative with broad barcode support [5]. It brings Java deployment and Python integration work. Choose it when an existing Java platform or a benchmark justifies that integration; this Python desktop project benefits from direct native bindings instead.

OpenCV-only decoding

Suitable for the barcode families its decoder supports, and for separate QR workflows. Its documented retail-format decoder is not a general PDF417 engine [3]. Retain OpenCV for preprocessing rather than treating it as a replacement for ZXing in this application.

ZBar / pyzbar

pyzbar documents a focus on one-dimensional barcodes and QR codes [4]. Do not assume PDF417 suitability from the existence of an API constant. Establish actual runtime support and benchmark the target format before considering it for this NID workflow.

Commercial PDF417 SDK

A licensed SDK is a possible second-stage evaluation if the open-source baseline misses required targets on difficult images. Verify its current format support, licensing, offline operation and data flow before adoption. No vendor accuracy claim or price has been relied on here; no commercial dependency is included.

OCR or a learned detector

OCR may separately recover visible printed fields, but cannot substitute for barcode byte recovery or authenticate the document. A learned barcode locator may improve crop proposals on cluttered scenes; it adds model assets, training data and evaluation work. Neither is needed to establish the initial baseline.

Decision rule

Keep ZXing-C++ unless a representative evaluation shows that another engine materially improves exact-payload recovery, latency or operational cost. Preprocessing is not universally helpful: thin bars may be damaged by filters, which is why the original grayscale candidate is tried first.

Validation & next decisions

Automated checks included

The test suite covers synthetic PDF417 codes at four right-angle rotations, a combined blur/noise/skew case, byte-preserving binary decoding, EXIF rotation, blank inputs, cancellation, crop and filter candidates, conservative parsing, CSV/JSON round-trip, deduplication and export failure/retry. These cases are reproducible and contain no real identity data.

A practical field benchmark

Build a consented, access-controlled sample set spanning relevant card generations, phones/cameras, distance, glare, rotation, perspective and wear. Maintain trusted expected barcode bytes where available. Separate tuning samples from a held-out evaluation set.

Measure exact-byte success rate, false decodes, no-read rate and median/95th-percentile latency. Report results by image condition and card group. Compare the baseline against preprocessing ablations and any alternative SDK under identical input and hardware conditions. Do not report a single accuracy number without the dataset definition.

Known limits before deployment

No guaranteed recovery from glare, missing bars, severe blur or extreme perspective. No validated official field mapping, signature validation, decryption or government lookup. Crops and skew corrections are heuristics. Session deduplication is not persistent. Real camera hardware behavior and real NID decoding need operator validation.

Primary references

1. ZXing-C++ project and supported formats

<https://github.com/zxing-cpp/zxing-cpp>

2. Official Python bindings and examples

<https://github.com/zxing-cpp/zxing-cpp/tree/master/wrappers/python>

3. OpenCV barcode recognition tutorial

https://docs.opencv.org/5.0/tutorials/objdetect/barcode_detect_and_decode/barcode_detect_and_decode.html

4. pyzbar project and documented scope

<https://github.com/NaturalHistoryMuseum/pyzbar>

5. Original Java ZXing project

<https://github.com/zxing/zxing>